

Lecture 07

הנדסה הפוכה של מערכות ידע גרפיות: מבוסס על Grphify ו-Obsidian

Lecture 07 - Reverse Engineering of Graph Knowledge Systems
with Grphify and Obsidian

הרצאה 07 | סיכום שיעור

ד"ר יורם סגל

כל הזכויות שמורות לד"ר יורם סגל ©

June 2026

גרסה 1.0

מסמך לימודי – סיכום הרצאה מוקלטת

מילות מפתח: Reverse Engineering, Obsidian, Grphify, הנדסה לאחור, תורת הגרפים, גרף ידע, Knowledge Graph, צמתים וקשתות, מרכזיות, Centrality, קהילות, Community Detection, Hubs, Bridges, צוואר בקבוק, Single Point of Failure, חיסכון בטוקנים, Token Efficiency, חלון קונטקסט, Context Window, RAG, Compaction, Lost in the Middle, מרחק וקטורי, קשרים אסוציאטיביים, Vault, graph.json, hot.md, index.md, Ambiguous, Inferred, Extracted, ארכיטקטורת תוכנה, CrewAI, LangGraph, סוכני AI, Confusion, BugsInPy, Matrix.

תוכן העניינים

1	פתיחה: מטרת ההרצאה ופילוסופיית הקורס: Introduction . .	4
2	הבעיה: צוואר הבקבוק של חלון הקונטקסט: The Context	
4	Window Bottleneck	4
2.1	מגבלת ה-RAG: מרחק וקטורי מול קשר אסוציאטיבי	4
3	הפתרון: ארכיטקטורת שלוש השכבות: The Three-Layer Architecture	5
4	Grphify מול Obsidian: חלוקת התפקידים: Grphify vs. Obsidian	6
4.1	החוזק של Grphify: ניתוח כמעט חינמי	6
4.2	החוזק של Obsidian: ויקיפדיה של הפרויקט	6
5	מושגי יסוד: זרימת המידע במערכת: Core Concepts	7
6	שלושת סוגי הקשרים בגרף: The Three Edge Types	8
7	לב הרעיון: חיסכון בטוקנים: Token Efficiency	8
7.1	הפתרון הגרפי: שליפה ממוקדת	9
8	תורת הגרפים: מרכזיות וקהילות: Centrality and Communities	9
8.1	מה מלמדת אותנו קהילה על הארכיטקטורה	10
9	בעיית "Lost in the Middle" וקומפקטציה: Lost in the Middle .	10
10	ניתוח ארכיטקטוני בפועל: קריאת הגרף: Reading the Graph .	11
10.1	גשרים: יתרונות וחסרונות	11
10.2	התאמה בין ה-PRD למימוש	11
11	המטלה (EX04): הנדסה לאחור עם סוכני AI: The Assignment	12

12	משימות הליבה	11.1
13	המאגרים המוצעים	11.2
	שיקולי תכנון, חיסכון ויצירתיות: Planning, Efficiency & Creativity	12
13		
14	תובנות נבחרות מן הדיון: Selected Q&A Insights	13
14	הקהילות הן קהילות של קבצים	13.1
14	בניית מערך נתונים לבדיקת סוכן	13.2
15	סיכום: Conclusion	14

1 פתיחה: מטרת ההרצאה ופילוסופיית הקורס: Introduction

ד"ר יורם סגל פתח את ההרצאה בהצבת ההקשר הרחב של הקורס. שוק העבודה בהייטק השתנה: חברות מחפשות כיום אנשי מקצוע בכירים (Seniors) בעלי ניסיון, ופחות מגייסות מתחילים (Juniors). בעידן של בינה מלאכותית וסוכנים אוטונומיים, העומס המוטל על כל מהנדס גדל פי כמה, ולכן הציפייה היא שבוגר טרי יידע כבר לחשוב ולעבוד כמו ארכיטקט.

המסר המרכזי של ההרצאה

מטרת השיעור היא להעניק כלי שמאפשר לקרוא, להבין ולשפר בסיסי קוד גדולים שלא הכרתם, ובזמן-לחוסך בין 70% ל-95% מן הטוקנים שאנו צורכים בעבודה עם מודלי שפה. הכלי הזה הוא **תורת הגרפים** ויישומה בהנדסה לאחר של תוכנה.

המרצה הדגיש עיקרון עבודה חשוב: בעת שימוש במודל שפה (LLM) יש תמיד להוסיף תרומה אישית – לשנות, להרחיב ולבדוק את התוצר – שכן זו הדרך היחידה ללמוד באמת ולהפוך את הידע לנכס מקצועי. רעיון זה מלווה את כל המטלה הנגזרת מן ההרצאה.

2 הבעיה: צוואר הבקבוק של חלון הקונטקסט: The Context Window Bottleneck

צוואר הבקבוק המרכזי של כל מודל שפה הוא חלון הקונטקסט (Context Window). כמות המידע שניתן להזין מוגבלת, בעוד שכמות המידע שאנו רוצים להזריק עצומה. בשיעורים קודמים נדון פתרון ה-RAG, אך גם לו יש מגבלה מהותית.

2.1 מגבלת ה-RAG: מרחק וקטורי מול קשר אסוציאטיבי

שיטת ה-RAG מסדרת את המידע לפי מרחקים וקטוריים: מילים בעלות משמעות דומה ממוקמות קרוב זו לזו. אולם ייצוג זה מזהה רק **דמיון נושאי**, ולא **קשר אסוציאטיבי**.

למה מרחק וקטורי אינו מספיק

נתבונן בקבוצת המילים: **סרט**, **פופקורן**, **חניה**. כל אדם רואה מיד את הסיפור המקשר ביניהן – יציאה לקולנוע. אך במרחב וקטורי של RAG אין כל קרבה בין "סרט" ל-"פופקורן" או ל-"חניה"; הוא יזהה רק ש"סרט" קרוב ל-"סדרת טלוויזיה". הקשר האסוציאטיבי – לב ההבנה האנושית – נעלם.

מסקנה: כדי לתפוס קשרים אסוציאטיביים בין יחידות מידע יש לעבור מייצוג וקטורי לייצוג גרפי.

3 הפתרון: ארכיטקטורת שלוש השכבות: The Three-Layer Architecture

המרצה הציג עולם הבנוי משלוש שכבות, שכל אחת ממלאה תפקיד מובחן בהמרת נתונים גולמיים לידע נווט:

טבלה 1: שלוש השכבות של מערכת הידע הגרפית

Role / התפקיד	Layer / השכבה
כל הדאטה הגולמית: קוד, מסמכים, וידאו, אודיו	הקבצים שלכם / Files
בונה אוטומטית גרף של קשרים בין הקבצים	Grphify
מציג ומנווט את הגרף בצורה ויזואלית וטקסטואלית	Obsidian

Grphify הוא כלי חינמי שניתן להריץ ישירות מ-CLI (למשל מתוך Claude) או דרך הפייפליין. הוא סורק את הקבצים, מאתר קשרים ביניהם, ומייצר גרף שמתואר בקובץ `graph.json`. מעליו פועל Obsidian, המספק ייצוג ויזואלי וטקסטואלי נוח לבני אדם ולסוכני AI כאחד.

4 Grphify מול Obsidian: חלוקת התפקידים: Grphify vs. Obsidian

4.1 החזק של Grphify: ניתוח כמעט חינמי

החזק המרכזי של Grphify הוא שהוא עובד כמעט ללא עלות בטוקנים. הסיבה פשוטה: אם פונקציה A קוראת לפונקציה B, נוצר קו בין שני צמתים – ולשם כך אין צורך כלל במודל שפה, אלא רק בניתוח קוד Python מבני (AST).

AST – ניתוח לוגי טהור

AST (Abstract Syntax Tree) מזהה את הלוגיקה של הקוד עצמו – מי קורא למי, אילו תנאים קיימים וכדומה. זו הסקה מבנית ודאית שאינה עולה טוקנים. רק כאשר רוצים לנתח גם קובצי וידאו, אודיו או תוכן סמנטי עמוק – מתחילים לשלם בטוקנים, ולכן הכלי שואל מראש מהו עומק החקירה הרצוי.

4.2 החזק של Obsidian: ויקיפדיה של הפרויקט

Obsidian מתמחה בקובצי Markdown ומפרק כל בעיה למבנה מסודר של קבצים מקושרים – בדומה למבנה של סקילים. התוצאה דומה ל**וויקיפדיה**: כל דף מכיל קישורים לדפים אחרים, וניתן לנווט בין רעיון לרעיון ללא הגבלה. הכלי גם מזהה בעצמו קשרים ונותן להם שמות ומשמעויות.

אנלוגיית הוויקיפדיה

כמו בוויקיפדיה, שבה כל קריאה מובילה ללינק ולינק נוסף "ללא סוף", Obsidian הופך את אוסף הקבצים שלכם למרחב ידע נווט: הוא סורק במקביל (Parallel Depth Scan), פותח כמה תהליכים בו-זמנית, ומאפשר עבודה אנושית נוחה מעל הגרף.

5 מושגי יסוד: זרימת המידע במערכת: Core Concepts

ל-Obsidian יש **כספת** (Vault) – נקודת הפתיחה של הפרויקט, המגדירה "מי במשחק". המשתמש מחליט מאיפה להתחיל לסרוק: פרויקט שלם או תיקייה ממוקדת (החלטה המשפיעה ישירות על צריכת הטוקנים). הזרימה היא: זריקת החומר → ניתוח ב-Grphify → יצירת הגרף (graph.json →) שליחה לכספת ה-Obsidian.

טבלה 2: מושגי ההיררכיה של מרחב הידע

מושג / Concept	משמעות / Meaning
Portfolio	תיק העבודות הראשי – כותרת-העל המתארת מה נכלל
Domain	תחום, כמו "מחלקה" באוניברסיטה (פייתון, LaTeX)
Project	היחידה האופרטיבית: קוד בפועל, PRD ותוכניות עבודה

שני דומיינים באותו פרויקט

פרויקט אחד עשוי להכיל שני דומיינים שונים: דומיין ה-Python של הסוכנים, ולצדו דומיין ה-LaTeX של מסמכי הפרויקט. שניהם חיים באותו עץ, אך הם תחומים נפרדים.

הסוד: גרף על הסקילים

החלק החשוב ביותר בכל קובץ הוא הכותרת שבראשו. אותו רעיון מוזרק לסקילים: כאשר מפעילים Grphify על אוסף הסקילים, כולם מתוארים כרשת קשרים. **זהו לב החיסכון בטוקנים** – ראו את הסעיף הבא.

6 שלושת סוגי הקשרים בגרף: The Three Edge Types

Grphify תמיד מציג צמתים, ובין הצמתים שלושה סוגי חיבורים, הנבדלים ברמת הוודאות שלהם:

טבלה 3: שלושת סוגי הקשתות בגרף ורמת הוודאות שלהם

Meaning / משמעות	סוג הקשר / Edge Type
“חקוק בסלע”: פונקציה קוראת לפונקציה – ודאות מלאה	Extracted
הסקה של ה-LLM מתוך הערות וטקסט – סבירות חלקית	Inferred
דו-משמעי: “משהו לא מריח טוב”, דורש בדיקה אנושית	Ambiguous

קשר דו-משמעי (Ambiguous)

Grphify עשוי לומר: “פונקציה זו שימשה פעם אחת לחישוב מחירי מניות, ופעם אחרת לספירת דפים בפרויקט LaTeX. איני בטוח – אולי לכך התכוונת, אבל כדאי שתבדוק.” זהו בדיוק המצב שבו נדרשת תשומת לב הנדסית.

7 לב הרעיון: חיסכון בטוקנים: Token Efficiency

המרצה הסביר מדוע עבודה רגילה מבזבזת טוקנים: לפני שכתבתם ולו מילה אחת בפרומפט, המערכת כבר לוקחת את כל הסקילים שלכם, שולפת את התיאורים (Descriptions) שלהם, ומזריקה אותם לראש חלון הקונטקסט. אם יש לכם עשרות או מאות סקילים, החלון מתמלא עוד לפני שהתחלתם.

הבזבוז הסמוי

ככל שמספר הסקילים גדל, נשאר פחות ופחות מקום לשאלה האמיתית. החלון מתמלא במידע שאינו רלוונטי לשאילתה הנוכחית – וזהו הבזבוז המרכזי של הטוקנים.

7.1 הפתרון הגרפי: שליפה ממוקדת

בשיטה הגרפית, המערכת קוראת תחילה את השאלתה, מבינה על מה מדובר, ובונה לעצמה גרף ממוקד. אם השאלה עוסקת בפרויקט תמחור, היא ניגשת ישירות לצומת "תמחור" בגרף, ושולפת במכה אחת את כל הצמתים שמתחתיו – ללא חיפוש בכלל הסקילים.

השוואת הגישות

RAG נאיבי מכניס רעש למערכת: סקיל אחר סקיל, מידע על מידע, עד שהחלון עמוס. הגישה הגרפית, לעומת זאת, שולפת רק את תת-העץ הרלוונטי. שתי הגישות הן דרכים שונות לתחזק מסד נתונים – כשם ש-RAG אינו שייך לחברת מודל מסוימת, כך גם הגישה הגרפית. אין כאן פגיעה בביצועים – להפך.

8 תורת הגרפים: מרכזיות וקהילות: Centrality and Communities

המרצה הדגיש שאין הוא מלמד את תורת הגרפים עצמה, אלא משתמש בה. שני מושגים מרכזיים ממנה משרתים את ניתוח הקוד:

מרכזיות (Centrality)

צומת בעל מרכזיות גבוהה הוא צומת שאליו מגיעים ויוצאים קשרים רבים. ככל שהעיגול בגרף גדול יותר, כך מצביעים אליו יותר קווים. קיימים אלגוריתמים ייעודיים לאיתור צמתים מרכזיים – "מנהיגים" בגרף.

קהילה (Community)

קהילה היא קבוצת צמתים שביניהם קשרים רבים בפנים ומעט "גשרים" החוצה. כאשר היחס בין הקשרים הפנימיים לחיצוניים גבוה – מדובר בקהילה. צומת (Node) יכול להיות קובץ, פונקציה, או כל יחידת מידע.

8.1 מה מלמדת אותנו קהילה על הארכיטקטורה

קהילה מלמדת על הארכיטקטורה הלוגית של המערכת. הדוגמה הטריטוריאליזם היא אפליקציית Full-Stack: קהילה אחת היא ה-Client Frontend וקהילה אחרת היא ה-Backend (Server). בתוך כל אחת יש קשרים רבים, וביניהן רק מעט חיבורים – שאלה ותשובה.

קהילה כדגל אדום ארכיטקטוני

דמיינו קובץ עם 50 פונקציות שכל אחת עושה דבר אחר ללא קשר. כקובץ, זהו צומת עם פונקציות רבות – אך אם רואים שהן "בורחות" החוצה אל קהילות אחרות, זו עדות לתכנון לקוי. כך הגרף חושף ארכיטקטורה לא תקינה.

בעבודת הנדסה לאחור, הצעד הראשון הוא מבט מאקרו: "מי נגד מי" – היכן השרת, היכן הלקוח – ולאחר מכן ביצוע Drill-Down: פתיחת כל קהילה לתת-קהילות, ועוד.

9 בעיית "Lost in the Middle" וקומפקטציה: Lost in the Middle

המרצה הדגיש תופעה קריטית: בחלון הקונטקסט קיימת "בעיית דיור". המודל מייחס משקל רב למידע שבתחילת הסשן ולמידע שבסופו, בעוד שהמידע **באמצע** נחלש ועלול להישכח או להשתבש.

המידע באמצע חלש

המערכת זוכרת היטב את ההתחלה ואת הסוף, אך "מאבדת" את האמצע. אחת הסיבות: לעיתים שואלים באמצע שאלה הסותרת שאלה קודמת, והמידע הסותר פוגע באיכות התשובה.

הכלל המעשי

למרות שכיום מציעים חלונות של מיליון טוקנים, האינטרס שלנו הוא שהסשנים יהיו **קצרים**. תמיד זכרו: **תחילת הסשן והמשפט האחרון שאתם כותבים הם החשובים ביותר**; כל היתר הוא תוספת. בתרגיל המלווה את ההרצאה תידרשו להוסיף **מדידת טוקנים** ולהוכיח חיסכון.

עבור סשנים גדולים מומלץ להפעיל את פעולת **הקומפקטציה** (Compaction): היא מסכמת את הסשן, מצמידה את ההתחלה לסוף, ו"זורקת" את האמצע – שכן ממילא ביצועי המודל שם חלשים. הפעולה לוקחת זמן אך מייעלת מאוד את העבודה.

10 ניתוח ארכיטקטוני בפועל: קריאת הגרף: Reading the Graph

תהליך הניתוח מתחיל במאקרו – מבט-על – ולאחר מכן מחפש קהילות, קשרים, והאבים (Hubs).

האב (Hub)

Hub הוא צומת שהכול עובר דרכו – צוואר בקבוק. Hub חלש (מעט מעברים) אינו מסוכן, אך Hub קריטי שדרכו זורם כל המידע הוא בעיה ארכיטקטונית.

10.1 גשרים: יתרונות וחסרונות

גשר (Bridge) מאפשר למידע להגיע משני מסלולים. היתרון: **יתירות** (Redundancy) ו-Fallback – אם מסלול אחד נכשל, המידע זורם באחר, וזו ארכיטקטורה רובסטית ואמינה יותר. החיסרון: **כפילות** (Duplication), שעלולה ליצור עומס ובעיות עקביות, שכן אותו מידע עשוי להגיע בזמנים שונים.

המיקום הפיזי חסר משמעות

זכרו באותיות של קידוש לבנה: **אין שום משמעות למיקום הפיזי של צומת בגרף**. קרבה או ריחוק על המסך הם עניין של נוחות תצוגה בלבד. המשמעות היחידה היא **האם יש קו, וכמה קווים**. גודל העיגול מעיד על כמות המידע או הקשרים, והצבע (או רקע עדין) מסמן קהילה.

10.2 התאמה בין ה-PRD למימוש

אחד הכוחות המופלאים של הגישה: בדיקה האם הקוד שנכתב תואם למסמכי התצורה (PRD). ייתכן ש-Claude "היה עצלן" ומימש פחות מהנדרש, או להפך – "הגדיל ראש" ומימש יותר. הגרף חושף את חוסר ההלימה בין התיעוד למימוש בפועל, כאשר אין מספיק קשרים בין דפי ה-PRD לבין דפי המימוש.

הארכיטקט אינו עובד ב-Real-Time

יש להריץ את Grphify מחדש לאחר כל שינוי מהותי – למשל באמצעות טריגר ב-GitHub בכל commit. הארכיטקט בודק את התוצר **לאחר** שהושלם, בדיוק כפי שבודקים תרגיל מוגש – לא תוך כדי כתיבה. בפרויקט קטן אין טעם בכך; בפרויקט גדול המיועד לפרודקשן – זו חובה.

Authentication כמסלול ולא כמילה

תהליך אימות מוצג כ**מסלול** שלם הנראה לעין: UI (Submit) → Controller → Validation → קריאה מ-Database → אישור מעבר. נקודת חיבור שכזו, אם היא Single Point of Failure, חושפת גם סיכון נפילה וגם בעיית עומסים פוטנציאלית.

11 המטלה (EX04): הנדסה לאחר עם סוכני AI: The Assignment

המטלה הנגזרת מן ההרצאה דורשת לקחת בסיס קוד Python **שאינכם מכירים**, לבצע לו הנדסה לאחר באמצעות גרפים, ולהבין מה הוא עושה ברמה הארכיטקטונית – לא מה כל שורה מחשבת, אלא כיצד תוכנן הקוד וכיצד הוא בנוי.

11.1 משימות הליבה

1. **שכפול הקוד** (Clone): הורדת מאגר נבחר מ-GitHub, או הבאת קוד אישי משמעותי (באישור המרצה).
2. **הפעלת Grphify**: יצירת הגרף, האינדקס, ודפי הניווט (hot.md ועוד), וייצוגם ב-Obsidian.
3. **הנדסה לאחר**: חילוץ **סכמת בלוקים** ארכיטקטונית וסכמת OOP של המחלקות. אין להסתפק בקוד שכבר מתועד – הערך הוא בחילוץ ההבנה.
4. **בניית סוכני AI**: סוכן מבוסס CrewAI או LangGraph לניתוח, איתור, ותיקון של **באגים ארכיטקטוניים** (כגון Single Point of Failure ועומסים).

5. **לולאת שיפור:** הסוכן מיישם המלצות (למשל פיצול מודול גדול), מריץ Grphify שוב, ובודק את התוצאה – עם תנאי עצירה מוגדר ובדיקת Unit Tests לאחר כל שינוי.

ריבוי סוכנים

לא די בסוכן אחד. עדיף כמה סוכנים, שכל אחד מומחה בתחומו: אחד מומחה ב-GitHub ובהורדת קוד, אחד בניתוח גרפים ואיתור בעיות, ואחד בכתיבת הקוד מחדש לפי הגרף. כאן נכנסת אורקסטרציית הסוכנים שנלמדה בשיעור הקודם.

11.2 המאגרים המוצעים

טבלה 4: מאגרי הקוד המוצעים למטלה, לפי רמת מורכבות

מאגר / Repository	אופי / Character
BugsInPy	באגים אמיתיים מהשטח – מורכב, דורש התאמת סביבה
מאגרי תרגול אקדמיים	קטעי קוד פשוטים יותר שנועדו לתרגול

שימו לב לסביבה

BugsInPy הוא מאגר אמיתי, ולכן חובה לעבוד בסביבה וירטואלית מותאמת. אם ההתקנה מסתבכת (דרייברים, ספריות) – אל תתעקשו; עברו למאגר פשוט יותר. בחירת המאגר אינה משפיעה על הציון; מה שמשפיע הוא מה למדתם והפקתם.

12 שיקולי תכנון, חיסכון ויצירתיות: Planning, Efficiency & Creativity

המרצה ביקש להוכיח **חיסכון בטוקנים**: להציג "לפני" (עבודה רגילה) מול "אחרי" (עם Grphify המזין סקילים לסוכן), כולל מספרי טוקנים. מי שלא הצליח לחסוך – יסביר מדוע.

התקציב והזמן

כל הכלים רצים **לוקאלי** – אלו כלים קטנים ורזים. לפי ניתוח המרצה, זוג שעובד באופן מרוכז יסיים את הפרויקט בכ-5 שעות עבודה. מי שמרגיש שהוא מתקרב לזמן זה ללא התקדמות – משהו בעייתי וכדאי לעצור ולחשוב. אין להפוך את המטלה לפרויקט גמר.

היצירתיות היא הלב

הבסיס כתוב לכולם; כל אחד יכול להזין את הדף למודל ולקבל פתרון. ההבדל האמיתי הוא מה **אתם** מביאים מעצמכם: אילו שימושים נוספים, הרחבות ותובנות. לדוגמה, Grphify יכול לשמש גם לניהול **צוותים**: ייצוג של R&D, HR, מחלקות ותפקידים כגרף חושף בעיות במבנה הארגוני.

13 תובנות נבחרות מן הדין: Selected Q&A Insights

בסיום ההרצאה התפתח דיון מעשי. שתי תובנות מרכזיות:

13.1 הקהילות הן קהילות של קבצים

הקהילות שמזהה Grphify הן קהילות של **קבצים**, והסוכנים משתמשים בהן. גם מסמך PRD המתאר סוכן יופיע בגרף – המערכת בונה את הכול לגרף, כולל זיהוי הקהילות.

13.2 בניית מערך נתונים לבדיקת סוכן

לשאלה כיצד בודקים שסוכן (למשל צ'אט-בוט) עובד טוב, הציע המרצה תהליך שיטתי לבניית **מערך נתונים** (Dataset):

1. ייצור 40000–50000 שאלות אפשריות באמצעות מודל שפה, עם תשובות ידועות.
2. הזרקת **שגיאות מכוונות**: בחירת אותיות אקראיות והחלפתן, כדי לדמות שגיאות כתיב, באחוזים הולכים וגדלים.
3. טיפול בריבוי שפות (למשל ערבית מדוברת, פלסטינאית, מצרית, סורית) – שכן לא ידוע מראש כיצד יפנה המשתמש.
4. בניית **Confusion Matrix**: כמה פעמים הסוכן צדק מול כמה טעה.

מתי "המכונה עובדת"

מגדילים בהדרגה את אחוז השגיאות עד שהביצועים קורסים (סביב 50% שגיאות הסוכן כבר לא יעמוד). כשמגיעים למספר הגיוני בטבלת הבלבול – יודעים שיש "מכונה עובדת" ואומדן אמין לרמת הסיבולת שלה לשגיאות.

14 סיכום: Conclusion

ההרצאה הציגה את תורת הגרפים ככלי כפול-תכלית: מחד, לבצע **הנדסה לאחור** ולהבין ארכיטקטורה של קוד לא מוכר בקנה מידה של עשרות ומאות אלפי שורות; ומאידך, **לחסוך טוקנים** בצורה דרמטית באמצעות שליפה ממוקדת מתוך גרף ידע במקום הצפת חלון הקונטקסט. שילוב Grphify (הבונה האוטומטי, החינמי כמעט) עם Obsidian (המנווט הוויזואלי) מאפשר לבוגר טרי לחשוב ולעבוד כמו ארכיטקט – וזו, לדברי ד"ר יורם סגל, היכולת שתבדל אתכם בשוק.